

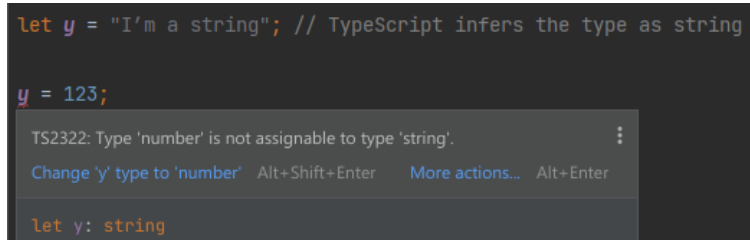
1

```
// Static typing for variables
let myNumber: number = 5;
let myString: string = "Hello, TypeScript";

// Type annotations for function parameters and return type
function addNumbers(a: number, b: number): number {
    return a + b;
}
```

2

```
let x = 10; // TypeScript infers the type as number
let y = "I'm a string"; // TypeScript infers the type as string
```



The screenshot shows a code editor with the following code:

```
let y = "I'm a string"; // TypeScript infers the type as string

y = 123;
```

A tooltip error message is displayed, stating: "TS2322: Type 'number' is not assignable to type 'string'." Below the message are two buttons: "Change 'y' type to 'number'" (with a keyboard shortcut of Alt+Shift+Enter) and "More actions..." (with a keyboard shortcut of Alt+Enter). At the bottom of the tooltip, the text "let y: string" is visible.

3

```
let z ;
z = "Test";
z = 123;
z = true;

console.log(z); // true
```

4

```
// Using interfaces to define object structure
interface Person {
    firstName: string;
    lastName: string;
    age: number;
}

// Function that takes a person object adhering to the Person interface
function getFullName(person: Person): string {
    return `${person.firstName} ${person.lastName}`;
}

let myPerson: Person = { firstName: "John", lastName: "Doe", age: 30 };
console.log(getFullName(myPerson)); // Output: John Doe
```

5

// Defining a simple enumeration

```
enum Days {  
  Monday,  
  Tuesday,  
  Wednesday,  
  Thursday,  
  Friday,  
  Saturday,  
  Sunday  
}
```

let today: Days = Days.Wednesday;

console.log(today); // Output: 2 (enums start from 0)

6

// Using classes and modules

```
class Animal {  
  private name: string;  
  
  constructor(name: string) {  
    this.name = name;  
  }  
  
  speak(): string {  
    return `${this.name} makes a sound`;  
  }  
}
```

// Exporting the Animal class from a module

export { Animal };

// Importing the Animal class in another file

import { Animal } from './animalModule';

let myAnimal = new Animal("Dog");

console.log(myAnimal.speak()); // Output: Dog makes a sound

9



7. JavaScript vs TypeScript

Feature	JavaScript	TypeScript
Type System	Dynamically-typed	Statically-typed (optional static typing)
Platform	Web browsers, server-side (Node.js)	Primarily for web development (JavaScript runtime)
Type Annotations	No explicit type annotations	Optional, allows explicit type annotations
Type Inference	Limited type inference	Powerful type inference system
Interfaces	No native support	Supports interfaces for defining object structure
Enums	No native support	Supports enums for named constant values
Classes and Modules	Supports, but with ES6 (ECMAScript 2015) syntax	Supports, with additional features for object-oriented programming
Compatibility	Works in any JavaScript runtime	Compatible with JavaScript; TypeScript code can be transpiled to JavaScript
Tooling Support	Varied; many code editors and IDEs available	Well-supported in Visual Studio Code and other popular editors
Development Process	Less tooling support for catching errors early	Enhanced tooling support for catching errors during development
Code Structure	Typically less structured due to dynamic nature	Supports a more structured and modular code organization
Adoption in Frameworks	Commonly used with frontend frameworks (React, Vue, Angular)	Widely used in Angular development, gaining popularity in other frameworks

Java vs TypeScript

Feature	Java	TypeScript
Type System	Statically-typed	Statically-typed (optional static typing)
Platform	Cross-platform	Primarily for web development (JavaScript runtime)
Compilation	Compiled to bytecode (JVM)	Transpiled to JavaScript
Usage	Enterprise applications, Android, backend	Web development, Node.js, frontend (Angular)
Object-Oriented	Purely object-oriented	Object-oriented (class-based, prototype-based)
Development Ecosystem	Mature ecosystem with extensive libraries and frameworks	Benefits from the JavaScript ecosystem, good library support
Tooling	IDEs like Eclipse, IntelliJ IDEA; Maven, Gradle for builds	Visual Studio Code, lightweight and powerful; good TypeScript support
Memory Management	Automatic garbage collection	Relies on JavaScript garbage collection

```
function test(name) {
    return `Hello ${name}`;
}

console.log(test("John"));
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\hp\Desktop\Ts Test> node app.js
Hello John
```

12

```
function test(name: string): string {  
    return `Hello ${name}`;  
}  
console.log(test("John"));
```

13



14

```
PS C:\Users\hp\Desktop\Ts Test> node ts-app.js  
Hello John
```

15

```
PS F:\GitHub MY\RAD-Practical1> tsc ts-app.ts  
tsc : File C:\Users\hp\AppData\Roaming\npm\tsc.ps1 cannot be loaded because running scripts is disabled on this system. For more information, see  
about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170.  
At line:1 char:1  
+ tsc ts-app.ts  
+ ~~~~  
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException  
+ FullyQualifiedErrorId : UnauthorizedAccess
```

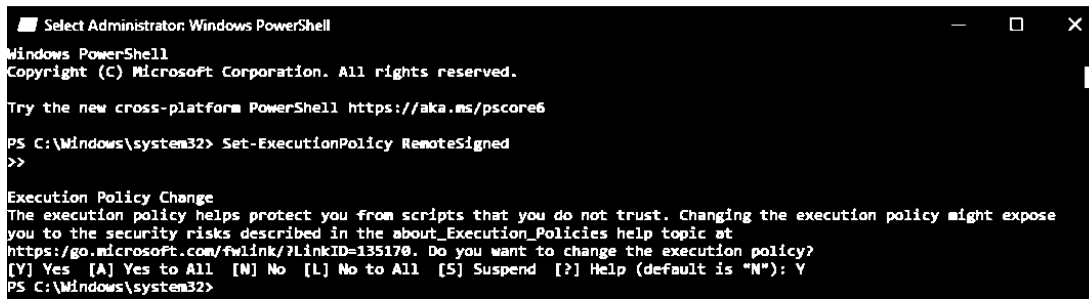
16

Run powershell as administrator



Type command :

```
Set-ExecutionPolicy RemoteSigned
```



Press 'Y' and enter

17

```
> node_modules
{} package-lock.json
{} package.json
```

18

```
> dist
> node_modules
v server
TS app.ts
{} package-lock.json
{} package.json
tsconfig.json
```

19

```
tsconfig.json 1 x
tsconfig.json > {} compilerOptions > outDir
51 |
52 |   > outDir 1 of 1
53 |   // "declarationMap": true,
54 |   // "emitDeclarationOnly": true,
55 |   // "sourceMap": true,
56 |   // "inlineSourceMap": true,
57 |   // "outFile": "./",
58 |   // "outDir": "./dist",
59 |   // "removeComments": true,
60 |   // "noEmit": true,
61 |   // "importHelpers": true,
62 |   // "importsNotUsedAsValues": "remove",
63 |   // "downlevelIteration": true,
64 |   // "sourceRoot": ""
```

20

```
package.json x
package.json > ...
1 | {
2 |   "devDependencies": {
3 |     "typescript": "^5.2.2"
4 |   },
5 |   "scripts": {
6 |     "start": "tsc && node dist/app.js"
7 |   }
8 | }
```

21

```
"scripts": {
  "start": "tsc && node dist/app.js"
}
```

22

```
v dist
JS app.js
```